I. Code Review & SOLID Principles

SOLID Principles

S - Single Responsibility Principle: one class should have only one responsibility.

O - Open/Closed Principle: software entities should be open for extension, but closed for modification.

L - Liskov Substitution Principle: a child class should be able to replace its parent without breaking the program.

I - Interface Segregation Principle: smaller, specific interfaces are better than one large interface.

D - Dependency Inversion Principle: high-level modules should not depend on low-level modules. Both should depend on abstractions.

====ItemController====

--Violation 1--

Which SOLID principle is violated?

Single Responsibility Principle

Where?

Controllers/ItemController.cs, method GetAll()

Why is it a violation?

The controller handles too many responsibilities: it receives HTTP requests, writes logs, retrieves data, converts data to a list, calculates statistics, and builds the response. Business logic such as calculating TotalCount and AverageValue should not be inside the controller.

Fix applied:

Moved the business logic to a service layer. The controller now only calls the service and returns the HTTP response.

--Violation 2--

Which SOLID principle is violated?

Single Responsibility Principle

Where?

Controllers/ItemController.cs, methods GetAll() and GetById()

Why is it a violation?

The controller uses `Console.WriteLine()` directly for logging. This mixes logging details with controller logic and makes the code harder to test and maintain.

Fix applied:

Replaced Console.WriteLine() with ASP.NET Core ILogger<ItemController> injected through the constructor.

--Violation 3--

Which SOLID principle is violated?

Dependency Inversion Principle

Where?

Controllers/ItemController.cs, field _reader and constructor

Why is it a violation?

The controller depends directly on `IItemReader`, which represents data access. The controller should depend on a higher-level service abstraction, not directly on the repository/data reader abstraction.

Fix applied:

Created a service interface, IGradeService, and injected it into the controller. The controller now depends on the business logic abstraction, while the service depends on the repository abstraction.

====ItemRepository====

--Violation 1--

Which SOLID principle is violated?

Single Responsibility Principle

Where?

Repositories/ItemRepository.cs, methods GetByIdAsync() and GetAllAsync()

Why is it a violation?

The repository has two responsibilities:

1. Managing access to stored items.

2. Deciding which items are valid/active for business use.

A repository should only handle data access. The rule that only active grades should be returned is business logic and should be placed in the service layer.

Fix applied:

Changed the repository methods to return raw data, then applied filtering in `GradeService`.

--Violation 2--

Which SOLID principle is violated?

Open/Closed Principle

Where?

Repositories/ItemRepository.cs, methods GetByIdAsync() and GetAllAsync()

Why is it a violation?

The repository filters only active items directly inside the data access methods:

„i => i.Id == id && i.IsActive”

and

„_items.Where(i => i.IsActive)”

If the filtering rule changes, the repository must be modified. For example, if the application later needs to filter passing grades, inactive grades, or archived grades, the repository code would need to change again.

Fix applied:

Moved filtering rules to the service layer, so the repository only retrieves data and the service decides how the data should be filtered.